

BASE DE DATOS PARA INGENIERIA



**BENEMÉRITA UNIVERSIDAD AUTÓNOMA DE
PUEBLA**

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

MANUAL TEORICO - PROYECTO FINAL

BRANDON RODRIGO MÉNDEZ MÉNDEZ

Yael TORCUATO SORIANO

MATRICULA: 202130905

MATRICULA: 202255251

DOCENTE: BEATRIZ MARTINEZ BELTRÁN

OTOÑO 2025

1. Planteamiento del Proyecto

Nombre del proyecto: Sistema de Gestión de Biblioteca.

El proyecto consiste en el desarrollo de un sistema que permita administrar los recursos de una biblioteca, incluyendo el registro de libros, usuarios y préstamos. El sistema debe almacenar la información en archivos, con el fin de tener un registro persistente de los datos.

Se gestionará la información detallada de los libros título, autor, ISBN, editorial, año de impresión, número de copias, etc., la información de los usuarios datos personales, inscripción y vigencia y el control de los préstamos qué libro se presta, a qué usuario y la fecha de devolución.

El sistema soluciona la necesidad de mantener un registro persistente y organizado de los datos, gestionando detalles críticos como el ISBN, editorial y copias de los libros, así como la vigencia de la inscripción de los usuarios. Además, automatiza reglas de negocio específicas, como el cobro de cuotas mensuales cuando un usuario intente solicitar un libro.

2. Objetivo del proyecto:

General:

Diseñar un sistema de gestión de biblioteca que permita administrar de manera eficiente los registros de libros, usuarios y préstamos, garantizando un control confiable de la información.

Específicos:

1. Registrar y mantener actualizada la información de los libros de la biblioteca.
2. Registrar y administrar la información de los usuarios inscritos, incluyendo su vigencia.
3. Controlar el préstamo de libros, verificando la vigencia de inscripción del usuario.
4. Calcular cuotas en caso de que la inscripción no esté vigente.
5. Almacenar la información de libros, usuarios y préstamos en archivos.

3. Requerimientos Funcionales

1. Gestión de Libros:

- Registrar libros con los siguientes datos: código, título, autor(es), ISBN, edición, editorial, año de impresión y número de copias.
- Consultar libros disponibles.
- Modificar información de libros existentes.

2. Gestión de Usuarios:

- Registrar usuarios con datos: nombre, dirección, teléfono, fecha de nacimiento, fecha de inscripción, fecha de vigencia.
- Verificar vigencia de la inscripción (1 año).
- Cobrar una cuota por cada mes vencido cuando la inscripción no esté vigente.

3. Gestión de Préstamos:

- Registrar préstamos de libros, vinculando usuario, libro y fecha de préstamo.
- Registrar devoluciones y actualizar disponibilidad del libro.
- Guardar la información de cada préstamo: qué libro se prestó, a qué usuario y fecha de entrega.

4. Persistencia de Datos:

- Guardar en archivos toda la información relacionada con libros, usuarios y préstamos.
- Cargar la información desde los archivos al iniciar el sistema.

4. Requerimientos No Funcionales

1. Seguridad:

- Validar que los usuarios tengan una inscripción vigente antes de permitir el préstamo.
- Verificar que los libros estén disponibles antes de prestarlos.

2. Usabilidad:

- Interfaz sencilla, con menús claros para registrar, consultar y modificar información.
- Mensajes de error y confirmación fáciles de entender.

3. Rendimiento:

- El sistema debe procesar consultas de libros y usuarios en menos de 3 segundos.
- Debe soportar al menos 500 registros de libros y 200 usuarios sin afectar el rendimiento.

4. Disponibilidad:

- El sistema debe funcionar localmente en todo momento, mientras el programa esté en ejecución.

5. Mantenibilidad:

- El código debe estar documentado y permitir futuras mejoras, como migrar de archivos a una base de datos relacional.

6. Escalabilidad:

- Posibilidad de ampliar en futuras versiones: integración de notificaciones por correo electrónico para devoluciones, o manejo de múltiples bibliotecas.

5. Diccionario de Datos

El diccionario de datos funciona como un repositorio central de metadatos que describe la estructura, organización y reglas de los datos almacenados. A continuación, se detalla la estructura de las tablas del sistema.

Tabla de usuarios				
Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
Id_usuario	Entero	----	PK, AUTOINCREMENT, NOT NULL	Identificador unico del usuario
nombre	Texto	50	NOT NULL	Nombre del usuario
correo	Texto	50	UNIQUE, NOT NULL	Correo del usuario
rol	Texto	20	NOT NULL (valores: estudiante, profesor, admin)	Rol dentro del sistema
contraseña	Texto	50	NOT NULL	Contraseña del usuario

Tabla de Libros				
Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
ISBN	Texto	20	PK, NOT NULL, UNIQUE	Codigo ISBN del libro
Titulo	Texto	150	NOT NULL	Titulo del libro
Autor	Texto	150	NOT NULL	Autor o autores del libro
Editorial	Texto	150	NULL	Editorial del libro
Año	Entero	---	NULL	Año de publicación
Categoria	Texto	50	NOT NULL	Categoria del libro
Disponible	Booleano	---	Default 1	Indica si está disponible (1) o no (0)
Edicion	texto	50	NOT NULL	Tipo de edicion de cada libro

Tabla de Prestamos

Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
Id_prestamo	Entero	---	PK, AUTOINCREMENT, NOT NULL	Identificador único del préstamo
Id_usuario	Entero	---	FK → Usuarios(id_usuario)	Usuario que realiza el préstamo
ISBN	Texto	20	FK → Libros(isbn)	Libro prestado
Fecha_prestamo	Fecha	---	NOT NULL	Fecha en que se prestó el libro
Fecha_devolucion	Fecha	---	NULL	Fecha límite de devolución
estado	Texto	20	NOT NULL (valores: activo, devuelto, vencido)	Estado del préstamo

Tabla de Reservas

Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
Id_reserva	Entero	---	PK, AUTOINCREMENT, NOT NULL	Identificador único de reserva
Id_usuario	Entero	---	FK → Usuarios(id_usuario)	Usuario que realiza la reserva
ISBN	Texto	20	FK → Libros(isbn)	Libro reservado
Fecha_reserva	Fecha	---	NULL	Fecha en que se realizo la reserva
estado	Texto	20	NOT NULL (valores: activo, devuelto, vencido)	Estado del préstamo

Tabla Recomendaciones

Campo	Tipo de Dato	Tamaño	Restricciones	Descripción
Id_recomendación	Entero	---	PK, AUTOINCREMENT, NOT NULL	Identificador único de la recomendación
Id_usuario	Entero	---	FK → Usuarios(id_usuario)	Usuario que recibe la recomendacion
ISBN	Texto	20	FK → Libros(isbn)	Libro recomendado
Motivo	Texto	500	NULL	Razón de la recomendación

6. Modelo Entidad-Relación (E-R)

Descripción del Modelo: El diagrama Entidad-Relación adjunto representa la estructura lógica y conceptual del sistema de biblioteca, detallando las entidades, sus atributos y las interacciones entre ellas.

Elementos del Diagrama

- **Entidades Principales (Rectángulos):**
 - **Usuario:** Eje central del sistema, caracterizado por atributos de control como ID Usuario, ID Cuota y MesesVencidos.
 - **Libro:** Recurso principal, con detalles bibliográficos como ISBN, Editorial, Edición y Año_Impresion.
 - **Autor:** Entidad separada para manejar la autoría múltiple, identificada por ID Autor.
- **Entidades Transaccionales:**
 - Prestamo: Registra la acción de prestar, controlando fechas (FechaPrestamo, FechaDevolucion) y el Estado.
 - Reserva: Gestiona las solicitudes pendientes, vinculando usuarios y libros mediante fechas y estados.
 - Recomendacion: Almacena sugerencias o feedback de los usuarios sobre los libros.
- **Relaciones y Cardinalidades (Rombos):**
 - **Solicita (Usuario 1:N Préstamo):** Un usuario puede realizar múltiples solicitudes de préstamo.
 - **Genera (Usuario 1:N Reserva):** Un usuario puede generar múltiples reservas de material.
 - **Recibe (Usuario 1:N Recomendación):** Relación que vincula las recomendaciones generadas por un usuario.
 - **Tiene (Reserva N:1 Libro):** Múltiples reservas pueden corresponder a un mismo libro (cola de espera).
 - **Incluye (Préstamo N:M Libro):** Relación que vincula los préstamos con los libros específicos involucrados.
 - **Puede (Libro N:M Autor):** Relación de muchos a muchos que permite que un libro tenga varios autores y un autor haya escrito varios libros.

7. Modelo Relacional

Descripción: El modelo relacional presentado a continuación traduce el diseño conceptual a un esquema de base de datos físico, definiendo las tablas, columnas y restricciones de integridad que conforman el sistema. Este modelo es la base directa para la creación del código SQL.

Componentes del Modelo:

1. Tablas Maestras (Catálogos):

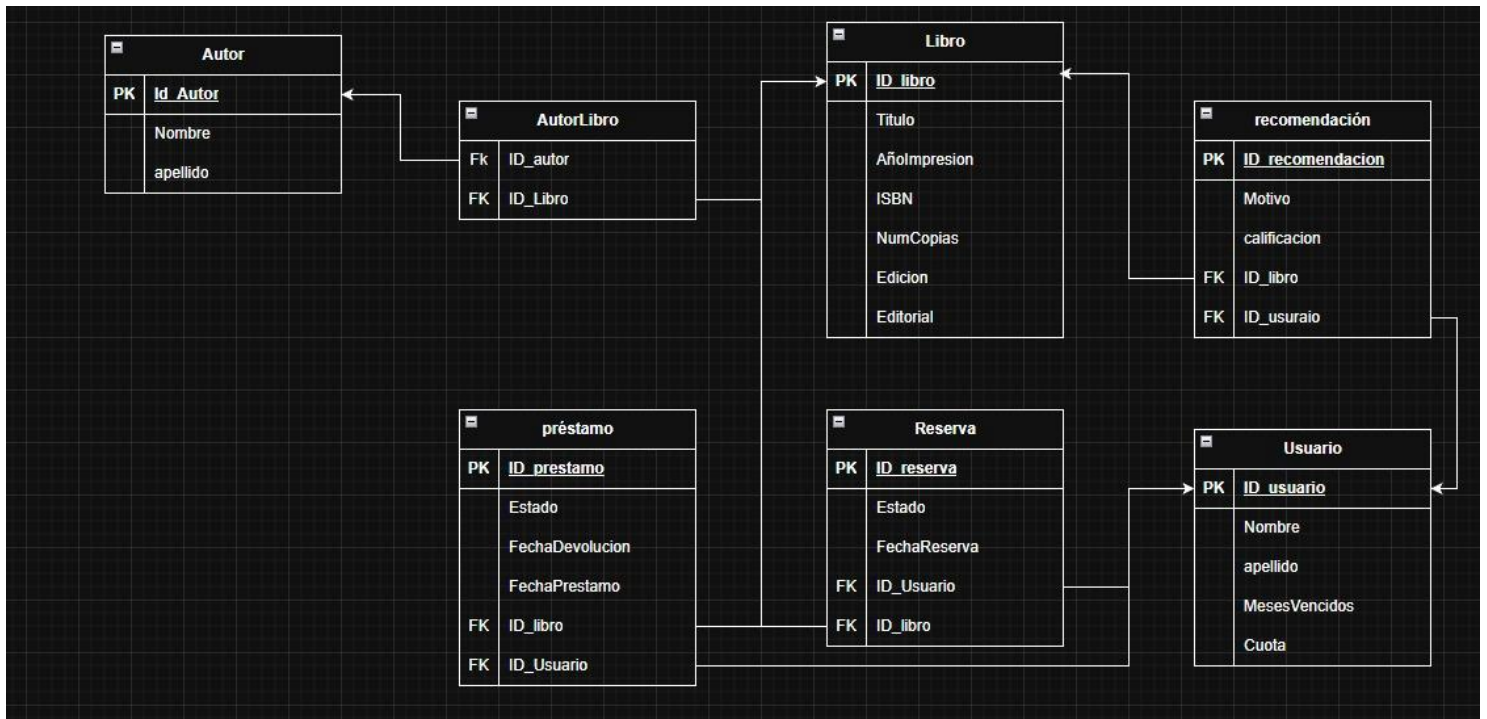
- **Libro:** Almacena los datos bibliográficos. Su llave primaria es ID_libro.
- **Usuario:** Almacena los datos de los socios. Su llave primaria es ID_usuario.
- **Autor:** Almacena los datos de los autores. Su llave primaria es Id_Autor.

2. Tablas Transaccionales y de Relación:

- **AutorLibro:** Tabla intermedia fundamental que resuelve la relación de muchos a muchos entre Autores y Libros. Contiene únicamente las llaves foráneas ID_autor y ID_Libro.
- **Préstamo:** Registra la circulación de material. Vincula un libro y un usuario mediante las llaves foráneas ID_libro y ID_Usuario, añadiendo atributos temporales como FechaPrestamo.
- **Reserva:** Funciona de manera similar a préstamos, vinculando ID_libro y ID_Usuario para gestionar la cola de espera.
- **Recomendación:** Permite registrar feedback, conectando nuevamente ID_libro y ID_usuario referenciado en el diagrama como ID_usuraio con una calificación y motivo.

Integridad Referencial: Las líneas que conectan las tablas representan las relaciones de llave foránea (FK). Estas aseguran que no se pueda crear un préstamo, reserva o recomendación para un usuario o libro que no exista previamente en las tablas maestras, garantizando la consistencia de los datos.

Modelo Relacional:



8. Normalización

Para garantizar la integridad de los datos, evitar redundancias y asegurar la escalabilidad del sistema, se aplicó un proceso de normalización riguroso a la base de datos, cumpliendo con las tres formas normales principales. Como se observa en el diagrama de normalización adjunto, la estructura resultante elimina datos repetidos y dependencias incorrectas.

Reglas de Normalización:

1. Regla de Atomicidad (Primera Forma Normal - 1FN):

- **Principio:** Cada campo debe contener un único dato indivisible atómico. No deben existir grupos repetitivos.
- **Aplicación en el proyecto:** Originalmente, un libro podía tener múltiples autores en una sola celda. Como se muestra en la normalización, esto se solucionó creando la "Tabla Autor" y una tabla intermedia "Tabla Autor libro". Esto garantiza que no haya apartados que pidan guardar más de un dato, cumpliendo la regla de atomicidad descrita en el diseño.

2. Dependencia de Llaves (Segunda Forma Normal - 2FN):

- **Principio:** En tablas con llaves compuestas, todos los atributos que no son llave deben depender de la llave completa, no solo de una parte.
- **Aplicación en el proyecto:** Esto es visible en la "Tabla Autor libro", donde la relación depende de la unión de ID_autor e ID_libro para ser única.

3. Regla de No Transitividad (Tercera Forma Normal - 3FN):

- **Principio:** Ninguna columna debe depender de otra columna que no sea la llave principal.
- **Aplicación en el proyecto:** En las tablas transaccionales como "Tabla préstamo" o "Tabla reserva", no se guardan datos del usuario como su nombre o dirección ni del libro (como su título), ya que estos dependerían transitivamente de la llave foránea. En su lugar, se almacenan únicamente las llaves foráneas FK ID_Usuario, FK ID_libro, asegurando que todos los atributos de la tabla dependan exclusivamente de la llave primaria del préstamo o reserva.

Esquema de Tablas Normalizadas:

A continuación, se presenta el diseño final de las tablas resultantes del proceso de normalización:

Tabla Autor

ID_Autor	Nombre	Apellido

Tabla Libro

ID_libro	Titulo	AñoImprecion	ISBN	NumCopias	Edicion	Editorial

Tabla Usuario

ID_usuario	Nombre	Apellido	MesesVencido	Cuota

Tabla Recomendación

ID_recomendación	Motivo	calificacion	FK_libro	FK_Usuario

Tabla préstamo

IDprestamo	Estado	Fecha Devolucion	Fecha Prestamo	FK ID_Usuario	FK ID_libro

Tabla reserva

ID_reserva	Estado	FechaReserva	FK ID_libro	FK ID_Usuario

Tabla Autor libro

ID_autor	ID_libro

9. Arquitectura del Sistema

El sistema fue desarrollado bajo el patrón de diseño MVC Modelo-Vista-Controlador adaptado para aplicaciones de escritorio. Esta arquitectura permite separar la lógica de negocio, la interfaz de usuario y el manejo de datos, facilitando la escalabilidad y el mantenimiento.

- **Modelo (Model):** Representa la estructura de los datos. Se implementa a través de Data Classes en Python y la estructura física en SQLite
- **Vista (View):** Es la capa de presentación. Desarrollada con Tkinter, gestiona las ventanas, formularios y tablas que interactúan con el usuario
- **Controlador (Controller):** Gestiona la lógica de negocio. Recibe las peticiones de la Vista, procesa los datos, validaciones, cálculos y se comunica con la Base de Datos.

10. Tecnologías Utilizadas

- Lenguaje de Programación: Python 3.12+
- Interfaz Gráfica (GUI): Tkinter - Biblioteca estándar de Python.
- Gestión de Base de Datos: SQLite3.
- Dependencias Externas: tkcalendar para el widget de calendario visual.
- Paradigma: Programación Orientada a Objetos.

11. Modelo de Datos

El sistema opera sobre una base de datos relacional SQLite normalizada:

- **Separación de entidades fuertes:** Libros, Usuarios, Autores.
- **Tablas transaccionales:** Prestamos, Reservas, Recomendaciones.
- **Tabla de ruptura (N:M):** Autor_Libro.
- **Tipos de Datos:** Se utiliza DATE para el manejo cronológico correcto de fechas, permitiendo ordenamientos y cálculos de vencimiento precisos.

12. Descripción Detallada de Archivos

A continuación, se describe la responsabilidad técnica de cada módulo del sistema:

Archivos Raíz y Configuración

main.py (Punto de Entrada):

- Archivo principal que arranca la aplicación.
- Inicializa la ventana raíz (Tk) e instancia la vista principal (MenuScreen).
- Mantiene el mainloop de la interfaz gráfica.

database.py:

- Clase ConexionDB: Gestiona la conexión física con biblioteca.db.
- Habilita soporte para llaves foráneas (foreign_keys).
- Contiene el método _crear_tablas(), que ejecuta el DDL (Data Definition Language) para generar la estructura normalizada si no existe.
- Provee métodos genéricos ejecutar_consulta SELECT y ejecutar instrucciones INSERT/UPDATE/DELETE.

```
class ConexionDB:
    def __init__(self):
        self.base_datos = "biblioteca.db"
        self.conexion = None
        self.cursor = None
        self._crear_tablas()

    def conectar(self):
        try:
            self.conexion = sqlite3.connect(self.base_datos)
            self.conexion.execute("PRAGMA foreign_keys = ON")
            self.cursor = self.conexion.cursor()
            return True
        except sqlite3.Error as e:
            messagebox.showerror("Error BD", f"Error al conectar: {e}")
            return False

    def desconectar(self):
        if self.conexion:
            self.conexion.close()

    def ejecutar_instruccion(self, sql, parametros=()):
        """Ejecuta INSERT, UPDATE, DELETE y retorna el ID"""
        if self.conectar():
            try:
                self.cursor.execute(sql, parametros)
                self.conexion.commit()
                return self.cursor.lastrowid or True
            except sqlite3.Error as e:
```

```

        messagebox.showerror("Error SQL", f"Error al ejecutar: {e}")
        return False
    finally:
        self.desconectar()
    return False

def ejecutar_consulta(self, sql, parametros=()):
    """Ejecuta SELECT"""
    datos = []
    if self.conectar():
        try:
            self.cursor.execute(sql, parametros)
            datos = self.cursor.fetchall()
        except sqlite3.Error as e:
            messagebox.showerror("Error SQL", f"Error al consultar: {e}")
        finally:
            self.desconectar()
    return datos

```

Está es la base de datos de nuestro sistema, con las cuatro operaciones para formar en CRUD y así poder gestionar nuestro sistema

13. Controladores

Contienen la lógica de negocio y actúan de intermediarios entre la BD y la Vista.

AdminLibros.py:

- Gestiona el CRUD de la tabla libros.
- **Método clave _procesar_autores:** Implementa la lógica para separar cadenas de texto de autores, buscar o crear los registros en la tabla autores y vincularlos en la tabla intermedia autor_libro.
- Maneja el inventario (disminuir_cantidad, aumentar_cantidad).

```

def _procesar_autores(self, id_libro, string_autores):
    """Maneja la lógica de la relación N:M entre Libros y Autores."""
    if not string_autores: return

    # 1. Limpiar relaciones anteriores
    self.db.ejecutar_instruccion("DELETE FROM autor_libro WHERE id_libro = ?", (id_libro,))

    lista_autores = string_autores.split(',')
    for autor_completo in lista_autores:
        autor_completo = autor_completo.strip()
        if not autor_completo: continue

    # Separar nombre y apellido para cumplir Atomicidad
    partes = autor_completo.split(' ', 1)
    nombre = partes[0]

```

```

apellido = partes[1] if len(partes) > 1 else " "

# 2. Verificar existencia o crear autor
sql_buscar = "SELECT id_autor FROM autores WHERE nombre=? AND apellido=?"
res = self.db.ejecutar_consulta(sql_buscar, (nombre, apellido))

if res:
    id_autor = res[0][0]
else:
    sql_insert_autor = "INSERT INTO autores (nombre, apellido) VALUES (?, ?)"
    id_autor = self.db.ejecutar_instruccion(sql_insert_autor, (nombre, apellido))

# 3. Crear relación en tabla intermedia
if id_autor:
    self.db.ejecutar_instruccion("INSERT INTO autor_libro (id_autor, id_libro) VALUES (?, ?)",
    (id_autor, id_libro))

```

La función más relevante de este controlador es `_procesar_autores`, la cual recibe la cadena de texto ingresada por el usuario y la descompone para gestionar las relaciones en la base de datos.

AdminUsuarios.py:

- Gestiona el CRUD de la tabla usuarios.
- Se encarga de mapear los datos atómicos (Nombre y Apellido por separado) hacia la base de datos.

```

def obtener_todos(self):
    sql = "SELECT * FROM usuarios"
    resultados = self.db.ejecutar_consulta(sql)
    usuarios = []
    for datos in resultados:
        # Mapeo manual de índices de columna a atributos del objeto
        usuario = Usuario(
            nombre=datos[1],
            apellido=datos[2], # Separación atómica
            meses_vencidos=datos[3],
            id_cuota=datos[4], # Mapeo de columna 'cuota'
            direccion=datos[5],
            # ... (resto de atributos)
            id_usuario=datos[0] # Llave primaria
        )
        usuarios.append(usuario)
    return usuarios

```

Este método reconstruye los objetos `Usuario` asignando cada índice de la columna de la base de datos al atributo correspondiente del modelo, manejando la distinción entre Nombre y Apellido para cumplir con la normalización.

AdminPrestamos.py:

- Orquesta la transacción de préstamo.
- Valida disponibilidad de stock antes de registrar.
- Gestiona el ciclo de vida: Al finalizar un préstamo, marca la fecha real de devolución y repone el stock del libro automáticamente.

```
def guardar(self, prestamo):
    # 1. Validación de Inventario (Orquestación con AdminLibros)
    if self.ctrl_libros.disminuir_cantidad(prestamo.libro):
        # 2. Si el inventario se descontó con éxito, registramos el préstamo
        sql = """
            INSERT INTO prestamos (id_libro, id_usuario, fecha_prestamo, fecha_limite,
            fecha_devolucion_real, estado)
            VALUES (?, ?, ?, ?, ?, ?)
            """
        return self.db.ejecutar_instruccion(sql, (
            prestamo.libro,
            prestamo.usuario,
            prestamo.fecha_salida,
            prestamo.fecha_limite,
            prestamo.fecha_devolucion_real,
            prestamo.estado
        ))
    # 3. Si no hay stock, retornamos False y no se registra nada
    return False
```

Esta función es la más relevante, ya que no trabaja de forma aislada. Si no que coordina la interacción entre el inventario de libros y el registro de préstamos para mantener la integridad de los datos.

AdminReservas.py:

- Gestiona la cola de espera de libros.
- Implementa la validación cruzada: Impide que un usuario reserve un libro si ya lo tiene prestado (verificar_estado_usuario_libro).

```
def verificar_estado_usuario_libro(self, id_libro, id_usuario):
    # 1. Verificar si ya lo tiene prestado físicamente
    sql_prestamo = "SELECT id_prestamo FROM prestamos WHERE id_libro = ? AND id_usuario = ? AND estado = 'Pendiente'"
    if self.db.ejecutar_consulta(sql_prestamo, (id_libro, id_usuario)):
        return "PRESTADO"

    # 2. Verificar si ya tiene una reserva activa en cola
    sql_reserva = "SELECT id_reserva FROM reservas WHERE id_libro = ? AND id_usuario = ? AND estado = 'Activa'"
    if self.db.ejecutar_consulta(sql_reserva, (id_libro, id_usuario)):
        return "RESERVADO"

    # 3. Si pasa ambas pruebas, el usuario es apto
    return "OK"
```

Este módulo administra la cola de espera para los libros. Su característica técnica más relevante es la implementación de reglas de negocio restrictivas que consultan múltiples tablas antes de autorizar una operación.

AdminRecomendaciones.py:

- Controlador ligero para gestionar el feedback de los usuarios (CRUD simple).

```
def obtener_todas(self):
    # Uso de concatenación (||) y JOINs para vista humana
    sql = """
        SELECT r.id_recomendacion, u.nombre || ' ' || u.apellido, l.titulo, r.motivo, r.calificacion
        FROM recomendaciones r
        JOIN libros l ON r.id_libro = l.id_libro
        JOIN usuarios u ON r.id_usuario = u.id_usuario
    """
    res = self.db.ejecutar_consulta(sql)
    lista = []
    for d in res:
        # Mapeo de resultados a objeto
        rec = Recomendaciones(id_rec=str(d[0]), usuario=d[1], libro=d[2], motivo=d[3])
        rec.calificacion = d[4]
        lista.append(rec)
    return lista
```

Este módulo permite capturar la retroalimentación cualitativa y cuantitativa de los usuarios respecto al acervo bibliográfico. Actúa como una entidad de relación pura que vincula un Usuario y un Libro con atributos descriptivos adicionales.

Modelos (/models)

Clases de tipo *Data Class* que representan la estructura de los objetos en memoria.

Libro.py: Define la estructura del libro (título, ISBN, año de impresión, copias).

```
from dataclasses import dataclass

@dataclass
class Libro:
    codigo: str
    titulo: str
    isbn: str
    edicion: str
    editorial: str
    anio_impresion: str
    num_copias: int
    autores: str = ""
    id_libro: int = None

    def to_csv(self):
        return f"{self.codigo},{self.titulo},{self.autores},{self.isbn},{self.edicion},{self.editorial},{self.anio_impresion},{self.num_copias}"
```

Incluye los atributos `anio_impresion` y `num_copias` para el control de inventario, este modelo mantiene un atributo `autores` exclusivamente para facilitar la visualización

Usuario.py: Define al socio (nombre, apellido, vigencia, cuotas).

```
from dataclasses import dataclass

@dataclass
class Usuario:
    # Atributos Atómicos (1FN)
    nombre: str
    apellido: str

    # Datos de contacto y membresía
    direccion: str
    telefono: str
    fecha_nacimiento: str
    fecha_inscripcion: str
    fecha_vigencia: str

    # Datos financieros
    id_cuota: str
    monto_cuota: str
    meses_vencidos: int

    # Identificador opcional (None al crear, Integer al leer de BD)
    id_usuario: int = None

    def to_csv(self):
        """Serialización para exportación rápida"""
        return f"{self.nombre},{self.apellido},{self.direccion},..."
```

Cumple con la Primera Forma Normal al separar nombre y apellido. Además, integra lógica de negocio al incluir campos para el control de cuotas y de vigencia

Prestamo.py: Modela la transacción (fechas, estado, IDs de relación).

```
from dataclasses import dataclass

@dataclass
class Prestamo:
    libro: str
    usuario: str
    fecha_salida: str
    fecha_limite: str
    fecha_devolucion_real: str = "---"
    estado: str = "Pendiente"
    id_prestamo: int = None
    id_libro_real: int = None
    id_usuario_real: int = None

    def to_csv(self):
        return f"{self.libro},{self.usuario},{self.fecha_salida},{self.fecha_limite},{self.fecha_devolucion_real},{self.estado}"
```

Implementa el manejo de datos para la visualización humana y atributos que almacenan las llaves foráneas necesarias para las operaciones de actualización en la base de datos.

Reserva.py: Modela la solicitud de apartado.

```
from dataclasses import dataclass

@dataclass
class Reserva:
    libro: str
    usuario: str
    fecha_reserva: str
    estado: str = "Activa"
    id_reserva: int = None
    id_libro_real: int = None
    id_usuario_real: int = None

    def to_csv(self):
        return f"{self.libro},{self.usuario},{self.fecha_reserva},{self.estado}"
```

Incluye un atributo de estado Activa o Cancelada que permite gestionar el flujo de la solicitud sin eliminar el registro histórico, maneja internamente los IDs relacionales.

Recomendacion.py: Modela la sugerencia y calificación.

```
from dataclasses import dataclass

@dataclass
class Recomendaciones:
    id_rec: any
    usuario: str
    libro: str
    motivo: str
    calificacion: int = 5

    def to_csv(self):
        return f"{self.id_rec},{self.usuario},{self.libro},{self.motivo},{self.calificacion}"
```

Almacena tanto un motivo como una calificación, sirviendo como métrica de calidad para la biblioteca.

Vistas (/views)

Interfaces gráficas construidas con Tkinter.

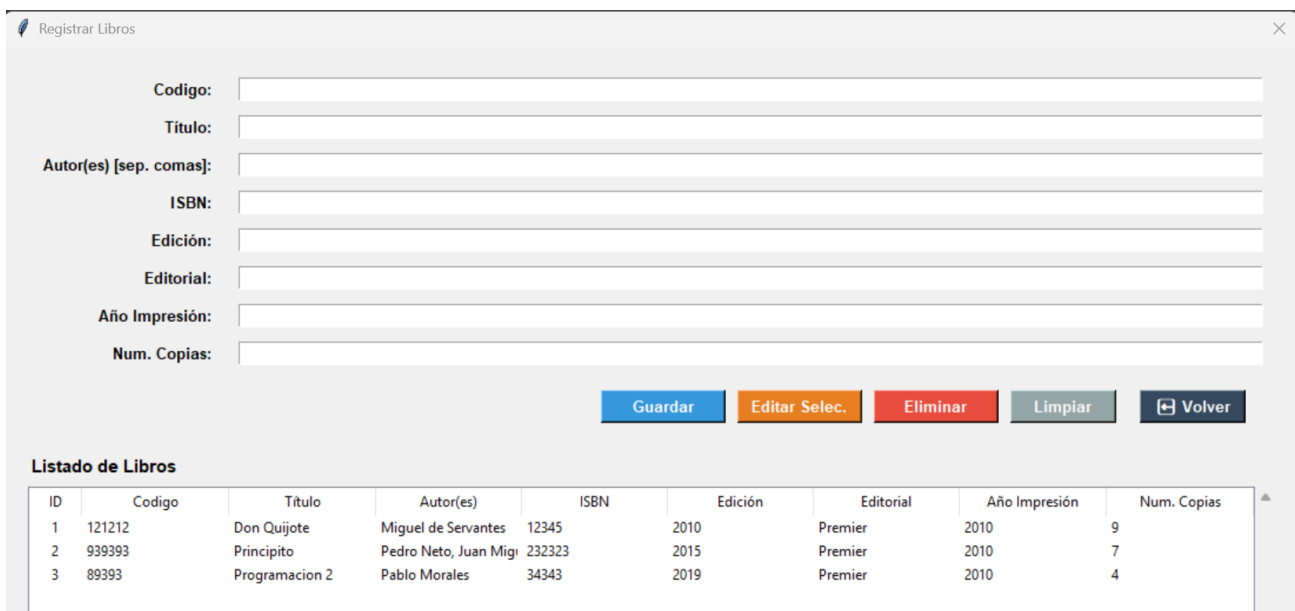
MenuScreen.py:

- Ventana principal tipo *Dashboard*. Contiene la navegación a los diferentes módulos.



LibrosScreen.py:

- Formulario para alta de libros y tabla de visualización. Muestra los autores registrados.



ID	Codigo	Título	Autor(es)	ISBN	Edición	Editorial	Año Impresión	Num. Copias
1	121212	Don Quijote	Miguel de Servantes	12345	2010	Premier	2010	9
2	939393	Principito	Pedro Neto, Juan Migi	232323	2015	Premier	2010	7
3	89393	Programacion 2	Pablo Morales	34343	2019	Premier	2010	4

UsuariosScreen.py:

- Formulario para registro de socios. Integra el componente SelectorFecha.

Registrar Usuarios

Nombre:

Apellido:

Dirección:

Teléfono:

Fecha nacimiento:

01

01

2025

Fecha inscripción:

01

01

2025

Fecha vigencia:

01

01

2025

ID Cuota:

Monto Cuota (\$):

0.00

Meses Vencidos:

0

Guardar

Editar Selec.

Eliminar

Limpiar

Volver

Listado de Usuarios

ID	Nombre	Apellido	Dirección	Teléfono	Fecha nacimiento	Fecha inscripción	Fecha vigencia	ID Cuota	Monto Cuota (\$)	Meses Vencidos
1	Brandon	Mendez	Av Reforma	2212313894	2003-03-13	2025-11-20	2025-12-30	1	100.00	1
2	Max	Gonzalez	Tepeaca 10	12121212	2003-05-05	2025-07-07	2025-12-05	2	100.00	5
3	Edgar	Morales	Av San Claudio	84848484	2004-06-04	2025-08-07	2025-12-09	3	1000.00	5

PrestamosScreen.py:

- Vista transaccional. Utiliza diccionarios en memoria para mapear los nombres seleccionados en los *Combobox* a sus respectivos IDs numéricos antes de enviar la transacción.

Gestión de Préstamos

Libro:

Usuario:

Fecha Limite:

02/12/2025

Realizar préstamo

Marcar como Devuelto

Actualizar Datos

Volver

ID	Libro	Usuario	Fecha Salida	Fecha Limite	Fecha Dev. Real	Estado
1	Programacion 2	Brandon Mendez	01/12/2025	31/12/2025	---	Pendiente
2	Don Quijote	Max Gonzalez	01/12/2025	17/12/2025	---	Pendiente

ReservasScreen.py:

- Vistas auxiliar para la gestión de apartados, siguiendo la misma lógica de mapeo de IDs que la pantalla de préstamos.

Gestión de Reservas (Normalizado)

Libro:

Usuario:

[Generar Reserva](#) [Cancelar \(Estado\)](#) [Editar](#) [Eliminar](#) [Limpiar](#) [Volver](#)

ID	Libro	Usuario	Fecha	Estado
1	Principito	Edgar Morales	01/12/2025	Activa

RecomendacionesScreen.py:

- Vistas auxiliar para la gestión de feedback, siguiendo la misma lógica de mapeo de IDs que la pantalla de préstamos.

Gestión de Recomendaciones (Normalizado)

Usuario:

Libro:

Motivo:

Calificación (1-5):

[Guardar](#) [Editar](#) [Eliminar](#) [Limpiar](#) [Volver](#)

ID	Usuario	Libro	Motivo	Calif
1	Brandon Mendez	Programacion 2	Es un libro muy interesante	5
2	Max Gonzalez	Principito	me gusto mucho	4
3	Edgar Morales	Don Quijote	no me gusto mucho	2